Review the user's question and the corresponding response using the additive 5-point scoring system described below. Points are accumulated based on the satisfaction of each criterion:

- Add 1 point if the response is relevant and provides some information related to the user's inquiry, even if it is incomplete or contains some irrelevant content.
- Add another point if the response addresses a substantial portion of the user's question, but does not completely resolve the query or provide a direct answer.
- Award a third point if the response answers the basic elements of the user's question in a useful way, regardless of whether it seems to have been written by an AI Assistant or if it has elements typically found in blogs or search results.
- Grant a fourth point if the response is clearly written from an AI Assistant's perspective, addressing the user's question directly and comprehensively, and is well-organized and helpful, even if there is slight room for improvement in clarity, conciseness or focus.
- Bestow a fifth point for a response that is impeccably tailored to the user's question by an AI Assistant, without extraneous information, reflecting expert knowledge, and demonstrating a high-quality, engaging, and insightful answer.

User: `<INSTRUCTION_HERE>`

<response>`<RESPONSE_HERE>`</response>

After examining the user's instruction and the response:

- Briefly justify your total score, up to 100 words.
- Conclude with the score using the format: "Score: <total points>"

Remember to assess from the AI Assistant perspective, utilizing web search knowledge as necessary. To evaluate the response in alignment with this additive scoring model, we'll systematically attribute points based on the outlined criteria.

Figure 2: **LLM-as-a-Judge prompt for our LLM to act as a reward model** and provide self-rewards for its own model generations. The model is initially trained with seed training data of how to perform well at this task, and then improves at this task further through our self-rewarding training procedure.

Section 2.2), following Xu et al. [2023], discarding the pair if their scores are the same. These pairs can be used for training with a preference tuning algorithm. We use DPO [Rafailov et al., 2023].

2.4 Overall Self-Alignment Algorithm

Iterative Training Our overall procedure trains a series of models $M_1, \ldots, M_T$ where each successive model t uses augmented training data created by the $t-1^{\text{th}}$ model. We thus define AIFT(M_t) to mean AI Feedback Training data created using model M_t.

Model Sequence We define the models, and the training data they use as follows:

M_0 : Base pretrained LLM with no fine-tuning.

M_1 : Initialized with M_0, then fine-tuned on the IFT+EFT seed data using SFT.

M_2 : Initialized with M_1, then trained with AIFT(M_1) data using DPO.

M_3 : Initialized with M_2, then trained with AIFT(M_2) data using DPO.

This iterative training resembles the procedure used in Pairwise Cringe Optimization and specifically is termed Iterative DPO, introduced in Xu et al. [2023]; however, an external fixed reward model was used in that work.

3 Experiments

3.1 Experimental Setup

Base Model In our experiments we use Llama 2 70B [Touvron et al., 2023] as our base pretrained model.

3.1.1 Seed Training Data

IFT Seed Data We use the human-authored examples provided in the Open Assistant dataset [Köpf et al., 2023] for instruction fine-tuning. Following Li et al. [2024] we use 3200 examples, by sampling only first conversational turns in the English language that are high-quality, based on their human annotated rank (choosing only the highest rank 0). In our experiments, we compare to a model fine-tuned from the base model using only this data via supervised fine-tuning, and refer to it as our *SFT baseline*.

EFT Seed Data The Open Assistant data also provides multiple ranked human responses per prompt from which we can construct evaluation fine-tuning data. We split this into train and evaluation sets, and use it to create LLM-as-a-Judge data. This is done by placing it in the input format given in Figure 2, which consists of the scoring criteria description, and the given instruction and response to be evaluated.[2] For training targets, chain-of-thought justifications and final scores out of 5 are not directly provided, so we use the SFT baseline to generate such output evaluations for each input, and accept them into the training set if the ranking of their scores agrees with the human rankings in the dataset. We resample the training set by discarding some of the data that receives the most common score so that the scores are not too skewed, as we observe many samples receive a score of 4. This results in 1,630 train and 541 evaluation examples (which do not overlap with the IFT data).

3.1.2 Evaluation Metrics

We evaluate the performance of our self-rewarding models in two axes: their ability to follow instructions, and their ability as a reward model (ability to evaluate responses).

Instruction Following We evaluate head-to-head performance between various models using GPT-4 [Achiam et al., 2023] as an evaluator over 256 test prompts (which we refer to as IFT test data) derived from various sources following Li et al. [2024] using the AlpacaEval evaluation prompt [Li et al., 2023]. We try the prompt in both orders comparing pairwise, and if the GPT-4 evaluations disagree we count the result as a tie. We also perform a similar evaluation with humans (authors). We additionally report results in the AlpacaEval 2.0 leaderboard format which is evaluated over 805 prompts, and compute the win rate against the baseline GPT-4 Turbo model based on GPT-4 judgments. Further, we report results on MT-Bench [Zheng et al., 2023b] a set of challenging multi-turn questions in various categories from math and coding to roleplay and writing, which uses GPT-4 to grade the model responses out of 10. Finally we also test the models on a set of 9 NLP benchmarks: ARC-Easy [Clark et al., 2018], ARC-Challenge [Clark et al., 2018], HellaSwag [Zellers et al., 2019], SIQA [Sap et al., 2019], PIQA [Bisk et al., 2020], GSM8K [Cobbe et al., 2021], MMLU [Hendrycks et al., 2021], OBQA [Mihaylov et al., 2018] and NQ [Kwiatkowski et al., 2019].

Reward Modeling We evaluate the correlation with human rankings on the evaluation set we derived from the Open Assistant dataset, as described in Section 3.1.1. Each instruction has on average 2.85 responses with given rankings. We can thus measure the *pairwise accuracy*, which is how many times the order of the ranking between any given pair agrees between the model's evaluation and the human ranking. We also measure the *exact match* count, which is how often the total ordering is exactly the same for an instruction. We also report the Spearman correlation and Kendall's τ. Finally, we report how often the responses that the model scores a perfect 5 out of 5 are rated as the highest ranked by humans.

[2]Note, the prompt, derived from Li et al. [2024], mentions "utilizing web search", but our model is not actually capable of this action.

3.1.3 Training Details

Instruction following training The training hyperparameters we use are as follows. For SFT we use learning rate $5.5e{-}6$ which decays (cosine) to $1.1e{-}6$ at the end of training, batch size 16 and dropout 0.1. We only calculate the loss on target tokens instead of the full sequence. For DPO we use learning rate $1e{-}6$ which decays to $1e{-}7$, batch size 16, dropout 0.1, and a β value of 0.1. We perform early stopping by saving a checkpoint every 200 steps and evaluating generations using Claude 2 [Anthropic, 2023] on 253 validation examples derived from various sources following Li et al. [2024]. This is evaluated pairwise against the previous step's generations using the AlpacaEval evaluation prompt format [Li et al., 2023].

Self-Instruction creation To generate new prompts we use a fixed model, Llama 2-Chat 70B with 8-shot prompting following Self-Instruct [Wang et al., 2023], where we sample six demonstrations from the IFT data and two from the model generated data, and use decoding parameters T = 0.6, p = 0.9. We use their prompt template for non-classification tasks and apply the same filtering techniques, including the ROUGE-L [Lin, 2004] similarity check, keyword filtering, and length filtering. Except for the prompt generation part, the other parts of the creation pipeline (generating the response, and evaluating it) use the Self-Rewarding model being trained. For candidate response generation we sample $N = 4$ candidate responses with temperature $T = 0.7$, $p = 0.9$. When evaluating candidate responses, as there is variance to these scores, in our experiments we also use sampled decoding (with the same parameters) and generate these evaluations multiple (3) times and take the average. We added 3,964 such preference pairs to form the AIFT(M_1) dataset used to train M_2 via DPO, and 6,942 pairs to form AIFT(M_2) used to train M_3.

3.2 Results

3.2.1 Instruction Following Ability

Head to head performance results are provided in Figure 3.

EFT+IFT seed training performs similarly to IFT alone We find that adding the Evaluation Fine-Tuning (EFT) task to training does not impact instruction following performance compared to using Instruction Fine-Tuning (IFT) data alone with an almost equal head to head (30.5% wins vs. 30.9% wins). This is a positive result because it means the increased capability of a model to self-reward does not affect its other skills. We can thus use IFT+EFT training as Iteration 1 (M_1) of our Self-Rewarding model, and then run further iterations.

Iteration 2 (M_2) improves over Iteration 1 (M_1) and SFT Baseline Iteration 2 of Self-Rewarding training (M_2) provides superior instruction following to Iteration 1 (M_1) with 55.5% wins for M_2 compared to only 11.7% for M_1 in a head to head evaluation. It provides similar gains over the SFT Baseline as well (49.2% wins vs. 14.5% wins). Clearly, there is a large jump in performance from M_1 to M_2 by using the preference data AIFT(M_1) provided by the reward model from Iteration 1.

Iteration 3 (M_3) improves over Iteration 2 (M_2) We see a further gain in Iteration 3 over Iteration 2, with 47.7% wins for M_3 compared to only 12.5% for M_2 in a head to head evaluation. Similarly, the win rate over the SFT Baseline for M_3 increases to 62.5% wins vs. 9.8%, i.e., winning more often than the M_2 model did. Overall, we see large gains from M_2 to M_3 through training using the preference data AIFT(M_2) provided by the reward model from Iteration 2.

Self-Rewarding models perform well on AlpacaEval 2 leaderboard We evaluate our models on the AlpacaEval 2.0 leaderboard format, with results given in Table 1. We observe the same findings as in the head-to-head evaluations, that training iterations yield improved win rates, in this case over GPT4-Turbo, from 9.94% in Iteration 1, to 15.38% in Iteration 2, to 20.44% in Iteration 3. Our Iteration 3 model outperforms many existing models in this metric, including Claude 2, Gemini Pro, and GPT4 0613. We show some